

第九章：類別與物件

一. 簡答題

2. 現在有一個 C#字串 str，請寫出字串方法 ToUpper()、Substring(2,4) 和 IndexOf("程式")的輸出結果，如下所示：

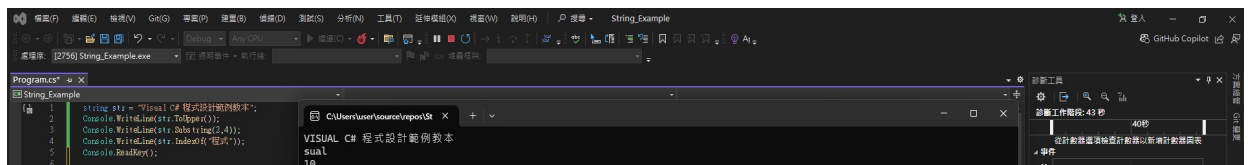
```
string str = "Visual C# 程式設計範例教本";
```

ToUpper: VISUAL C# 程式設計範例教本

Substring: sual

IndexOf: 10

```
string str = "Visual C# 程式設計範例教本";  
Console.WriteLine(str.ToUpper());  
Console.WriteLine(str.Substring(2,4));  
Console.WriteLine(str.IndexOf("程式"));  
Console.ReadKey();
```



6. 請簡單說明什麼是搜尋與排序?請問搜尋方法依照搜尋的資料可以分為哪兩種?

「搜尋」與「排序」在資料科學中都屬於資料結構與演算法的範疇。
電腦有相當多執行時間都在處理資料排序、搜尋，
兩者實際應用在資料庫系統、編譯器和作業系統之中。

可分為以下兩種：① 沒有排序的資料

② 已經排序的資料

1. 請簡單說明物件導向的應用程式開發和傳統應用程式開發的差異？

物件導向應用程式是將資料和操作一起思考，主要工作找出參與物件和其他物件之間的關係，而傳統應用程式是將資料和操作分開思考，著重於如何找出解決問題的演算法來建立程序或函數。

6. 請舉例說明 `private`、`protected` 和 `public` 三種修飾子的用途和差異？
什麼是「工具方法」(Utility Methods)？

`private`、`protected`、`public` 三種修飾子可用來指定成員層級

(嚴) `private`：成員變數或方法只能在類別本身呼叫或存取，預設走`private`。

↑ `protected`：成員變數或方法可以在類別本身和其子類別呼叫或存取，帶有繼承關係。

(鬆) `public`：成員變數或方法是此類別對外的使用介面，任一皆可呼叫及存取。

工具方法：在一個類別中，僅提供在同-類別成員方法呼叫存取。

二. 實作題

2. 請建立 C# 應用程式宣告 5 個元素的一維陣列後，使用亂數類別來產生陣列元素值，其範圍是 1~200 的整數，然後將陣列內容排序後，顯示在標籤控制項。

namespace Random_Array

{

public partial class Form1 : Form

{

public Form1()

{

InitializeComponent();

}

public void bubbleSort(int[] data)

{

try

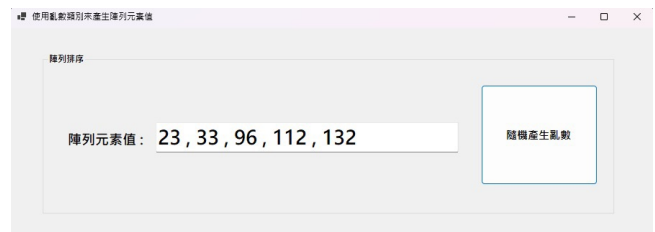
{

int len = data.Length;

for (int i = 0; i < len - 1; i++)

{

for (int j = 0; j < len - i - 1; j++)



```

        {
            if (data[j + 1] < data[j])
            {
                int temp = data[j + 1];
                data[j + 1] = data[j];
                data[j] = temp;
            }
        }
    }
}
catch (Exception ex)
{
    MessageBox.Show("排序發生錯誤: " + ex.Message);
}
}

```

```

private void button1_Click(object sender, EventArgs e)
{
    try
    {
        Random random = new Random();

        int[] num = new int[5];
        for (int i = 0; i < num.Length; i++)
        {
            num[i] = random.Next(1, 200);
        }

        int[] arrayToSort = num.ToArray();
        bubbleSort(arrayToSort);

        textBox1.Text = string.Join(" ", arrayToSort);
    }
    catch (Exception ex)
    {
        MessageBox.Show("執行發生錯誤: " + ex.Message);
    }
}

```



```
private void btnFindMinMax_Click(object sender, EventArgs e)
{
    try
    {
        int[] numbers = new int[6];

        numbers[0] = int.Parse(txtNumber1.Text);

        numbers[1] = int.Parse(txtNumber2.Text);

        numbers[2] = int.Parse(txtNumber3.Text);

        numbers[3] = int.Parse(txtNumber4.Text);

        numbers[4] = int.Parse(txtNumber5.Text);

        numbers[5] = int.Parse(txtNumber6.Text);


        // 呼叫函數計算最小值和最大值

        int minValue = arrMin(numbers);

        int maxValue = arrMax(numbers);


        // 顯示結果

        lblResult1.Text = $"最小值: {minValue}";

        lblResult2.Text = $"最大值: {maxValue}";

    }

    catch (Exception ex)

    {

        MessageBox.Show("請確認輸入的值是有效的整數。 \n" +
ex.Message, "錯誤", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

}

}

// arrMin 函數

private int arrMin(int[] arr)

{

return arr.Min();

}

// arrMax 函數

private int arrMax(int[] arr)

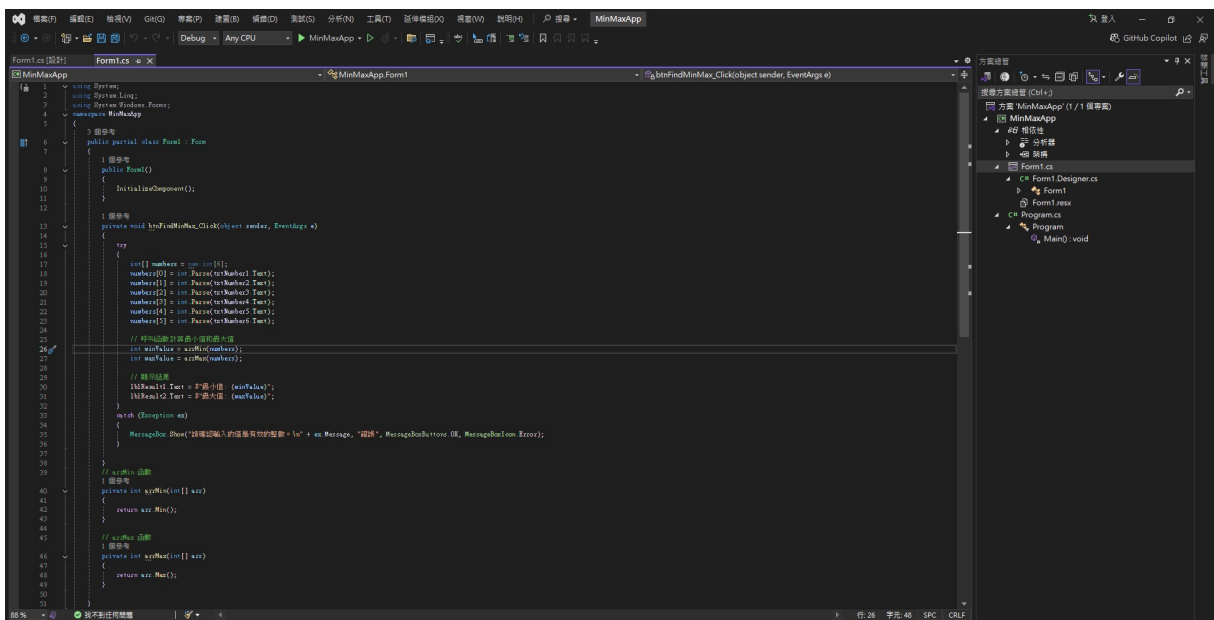
{

return arr.Max();

}

}

}



2. 請使用 C# 語言寫出 Box 類別的宣告來建立盒子物件，在類別提供計算盒子的體積與面積，並且繪出 Box 類別的 UML 類別圖，如下所示：

- 成員變數：Width、Height 和 Length 儲存寬、高和長。
- 建構子：Box(double width, double height, double length)。
- 成員方法：double Volume() 計算體積和 double Area() 計算面積。

```
using System;
```

```
public class Box
```

```
{
```

```
    // 屬性
```

```
    public double Length { get; set; }
```

```
    public double Width { get; set; }
```

```
    public double Height { get; set; }
```

```
    // 建構函式
```

```
    public Box(double length, double width, double height)
```

```
    {
```

```
        Length = length;
```

```
        Width = width;
```

```
        Height = height;
```

```
    }
```

```
    // 計算體積的方法
```

```
    public double CalculateVolume()
```

```
    {
```

```
        return Length * Width * Height;
```

```
    }
```

```
    // 計算表面積的方法
```

```
    public double CalculateSurfaceArea()
```

```
    {
```

```
        return 2 * (Length * Width + Length * Height + Width * Height);
```

```
    }
```

```
    // 顯示盒子的資訊
```

```
    public void DisplayBoxInfo()
```

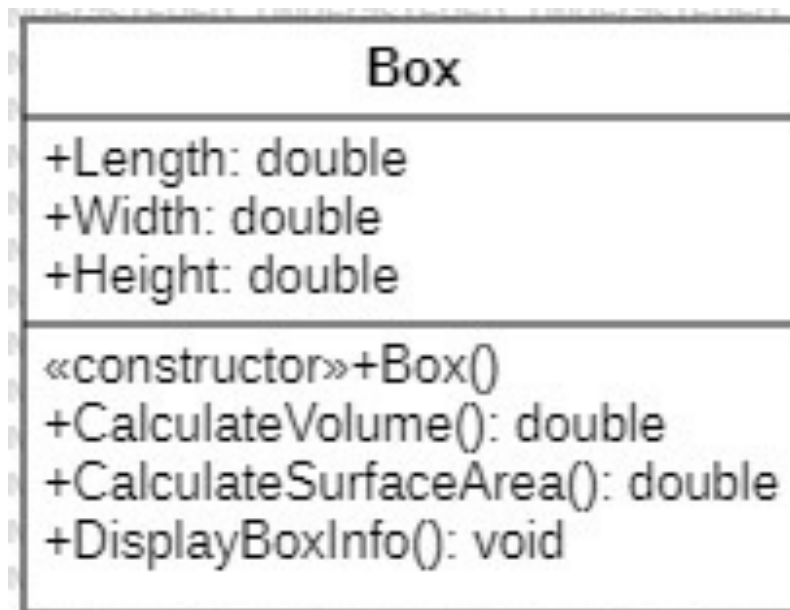
```

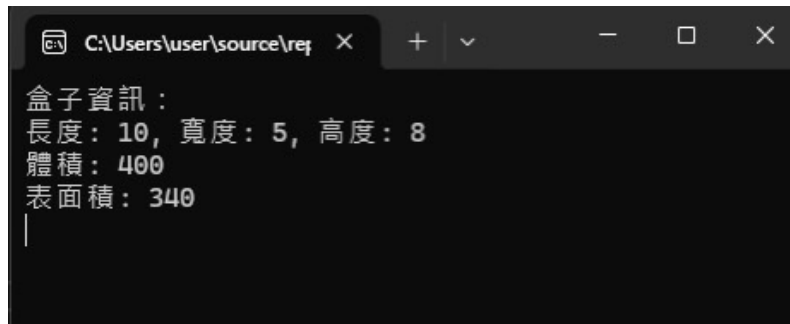
    {
        Console.WriteLine($"盒子資訊：");
        Console.WriteLine($"長度: {Length}, 寬度: {Width}, 高度:
{Height}");
        Console.WriteLine($"體積: {CalculateVolume()}");
        Console.WriteLine($"表面積: {CalculateSurfaceArea()}");
    }
}

class Program
{
    static void Main(string[] args)
    {
        // 建立 Box 物件
        Box myBox = new Box(10, 5, 8);

        // 顯示盒子的資訊
        myBox.DisplayBoxInfo();
        Console.ReadKey();
    }
}

```



A screenshot of a Windows command prompt window. The title bar shows the file path 'C:\Users\user\source\ref' and standard window controls. The command prompt displays the following text: '盒子資訊：', '長度：10, 寬度：5, 高度：8', '體積：400', and '表面積：340'. A cursor is visible on the line following '表面積：340'.

4. 請建立名片資料的 Cards 類別，擁有 Name、Occupation、Age、Phone 和 Email 成員變數儲存姓名、職業、年齡、電話和電子郵件資料，其中的 Phone 變數是參考另一個類別 PhoneList 的實例，PhoneList 類別擁有成員變數 HomePhone、BusinessPhone 和 CellPhone 儲存住家、公司和手機電話，最後建立 GetCard() 方法取得名片資料。

```
using System;
```

```
public class PhoneList
```

```
{
```

```
    // 成員變數：住家電話、公司電話和手機電話
```

```
    public string HomePhone { get; set; }
```

```
    public string BusinessPhone { get; set; }
```

```
    public string CellPhone { get; set; }
```

```
    // 建構函式
```

```
    public PhoneList(string homePhone, string businessPhone, string  
cellPhone)
```

```
    {
```

```
        HomePhone = homePhone;
```

```
        BusinessPhone = businessPhone;
```

```
        CellPhone = cellPhone;
```

```
    }
```

```
    public override string ToString()
```

```
    {
```

```
        return $"住家: {HomePhone}, 公司: {BusinessPhone}, 手機:
```

```
{CellPhone}";  
    }  
}
```

```
public class Cards
```

```
{  
    // 成員變數：姓名、職業、年齡、電話（參考 PhoneList 類別）、電子郵件
```

```
    public string Name { get; set; }  
    public string Occupation { get; set; }  
    public int Age { get; set; }  
    public PhoneList Phone { get; set; }  
    public string Email { get; set; }
```

```
    // 建構函式
```

```
    public Cards(string name, string occupation, int age, PhoneList phone,  
string email)
```

```
    {  
        Name = name;  
        Occupation = occupation;  
        Age = age;  
        Phone = phone;  
        Email = email;  
    }
```

```
    // GetCard 方法：取得名片資料
```

```
    public string GetCard()  
    {  
        return $"姓名: {Name}\n 職業: {Occupation}\n 年齡: {Age}\n 電話:  
{Phone}\n 電子郵件: {Email}";  
    }  
}
```

```
class Program
```

```
{  
    static void Main(string[] args)  
    {  
        // 建立 PhoneList 實例
```

```
PhoneList phoneList = new PhoneList("02-12345678", "03-98765432",  
"0912-345678");
```

```
// 建立 Cards 實例
```

```
Cards myCard = new Cards("Henry", "工程師", 30, phoneList,  
"henry@example.com");
```

```
// 顯示名片資料
```

```
Console.WriteLine(myCard.GetCard());
```

```
Console.ReadKey();
```

```
}
```

```
}
```

